CASE STUDY / 2026

GrowMore Reporting

Multi-tenant commerce analytics - from APIs to tested Silver data.

Python/dlt | dbt | Supabase PostgreSQL | FastAPI | Next.js

PREPARED BY

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	Platform Architecture	Page 4
03	Tenant Data Model	Page 5
04	Technology Stack	Page 6
05	The Engineering Challenge	Page 7
06	Approach & Quality Contracts	Page 8
07	Code Spotlight	Page 9
08	Verified Delivery	Page 10
09	Outcome & Next Stage	Page 11

This case study documents the implemented GrowMore reporting platform: tenant-aware ingestion, schema-per-client storage, reusable dbt transformations, data-quality contracts, and the current application boundary. Verified components are separated from planned Gold marts and live reporting.

SECTION 01

Project Overview

A multi-source, multi-tenant analytics foundation for commerce and marketing reporting.

GrowMore Reporting consolidates operational and marketing data from Shopify, Meta Ads, Instagram, Noon, and Klaviyo into a tenant-isolated PostgreSQL analytics environment. Python/dlt pipelines land source records into Bronze tables; dbt standardizes them into documented Silver views with stable keys, types, grains, and relationship tests.

The platform is designed for an agency operating many independent client accounts. Each client owns a stable schema, each source account is registered centrally, and every ingestion run resolves its destination from trusted metadata rather than user input. FastAPI and Next.js provide the secure application boundary, while Supabase JWT, PostgreSQL memberships, and Redis-backed authorization support tenant-aware access.

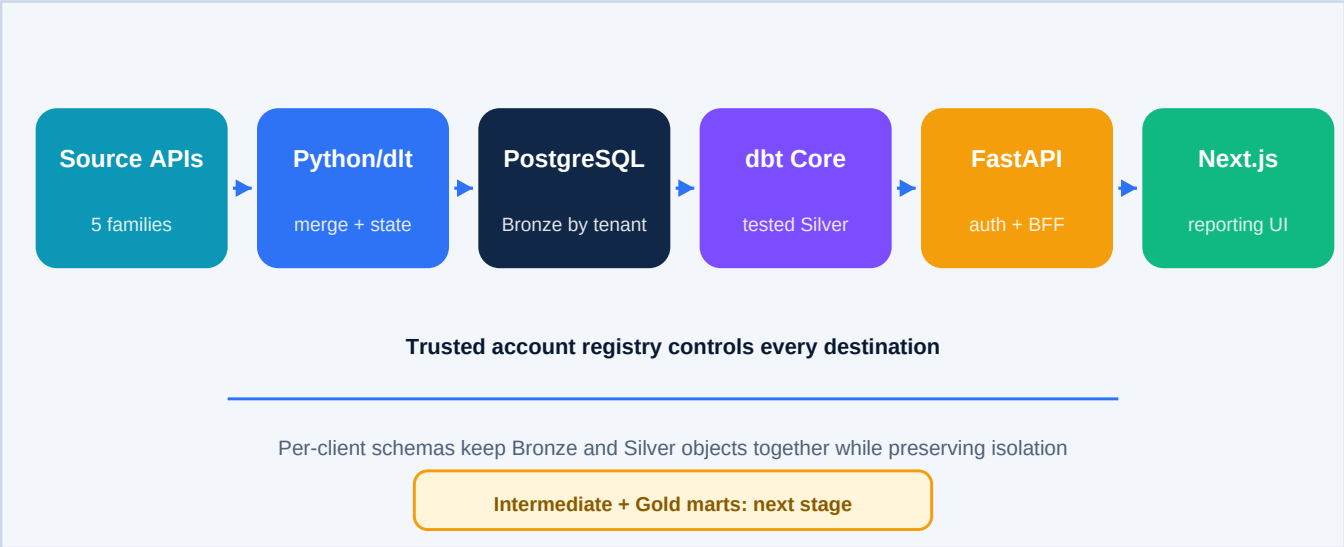
The delivered milestone is deliberately precise: the Silver layer covers the current Bronze sources and accounts. Intermediate models, Gold marts, production scheduling, and live reporting APIs remain the next stage and are not presented here as completed work.

5	~178	1,420	Tenant
Integrated source families	Silver views documented	dbt data tests documented	Schema-isolated architecture

SECTION 02

Platform Architecture

Source APIs move through controlled ingestion and transformation before application access.



Every layer has a narrow responsibility. dlt handles extraction state and idempotent merges. PostgreSQL enforces client boundaries and records sync outcomes. dbt converts heterogeneous Bronze tables into consistent Silver contracts. The application tier is intentionally downstream of those contracts, so dashboards never need to interpret raw vendor payloads.

SECTION 03

Tenant Data Model

A schema-per-client boundary with account-prefixed tables and reusable transformation logic.

BRONZE

Source-faithful tables

dlt merge loads, extraction metadata, sync outcomes

SILVER

Clean analytical contracts

Typed fields, declared grain, stable keys, tests

GOLD

PLANNED

Business marts

Agreed KPIs, currency rules, dashboard-ready tables

Reusable onboarding pattern. Transformation SQL lives once in a macro. A new account adds a registry row and a thin model stub that supplies the account code, schema, and alias. The same logic is therefore applied consistently across tenants without copying business SQL.

Registry

Trusted client/account mapping

Stable keys

source_key:id identity

Views

Silver evaluates from Bronze

RLS + JWT

Application authorization

SECTION 04

Technology Stack

A Python-first data platform with warehouse transformations and a secure web application boundary.

Layer	Technology & implemented responsibility
Source ingestion	Python + dlt - API extraction, merge disposition, state, retries and account-scoped datasets
Storage	Supabase PostgreSQL - stable client schemas, Bronze records, Silver views, metadata and sync tracking
Transformation	dbt Core - registry-driven macros, typed Silver models, generic tests and relationship checks
Backend	FastAPI + SQLAlchemy - Supabase JWT verification, tenant membership, BFF sessions and admin APIs
Shared state	Redis - rate limiting, authorization cache and session revocation across application instances
Frontend	Next.js + React + TypeScript - responsive reporting UI, secure server-side routes and CSV export
Runtime	Docker Compose - reproducible application services with environment-based configuration
Quality	pytest, Ruff, TypeScript checks and dbt data tests with explicit warning/error severity

Why this stack. dlt keeps ingestion lightweight enough for modest infrastructure, dbt makes transformation behavior reviewable in SQL, and PostgreSQL keeps tenant data and application metadata within one governed platform. FastAPI and Next.js consume curated contracts rather than raw payloads.

SECTION 05

The Engineering Challenge

Unify incompatible commerce and marketing APIs without weakening client isolation or data meaning.

The difficulty is not simply moving rows. Each platform exposes different pagination, history limits, nested structures, identifiers, money conventions, and grains. One source reports daily ad metrics; another exposes orders, line items, refunds, inventory, or event streams. Optional fields can disappear entirely when a source returns only nulls.

At the same time, the platform serves unrelated clients. A single routing defect could place records in the wrong schema, and a careless dashboard query could expose customer or campaign data across tenants. The design therefore treats destination resolution, row identity, PII handling, and authorization as data-engineering concerns - not just application concerns.

Schema drift

Guard optional fields and produce a stable Silver shape even when vendor payloads differ.

Mixed grains

Define keys for orders, line items, ad-day facts, media metrics and finance transactions.

Money semantics

Preserve currencies and distinguish major units from minor units before aggregation.

API resilience

Chunk large windows, resume report jobs and record blocked, empty or failed entities.

Tenant safety

Resolve destinations from trusted registry data and fail closed on mapping mismatches.

Privacy

Minimize customer PII by default and keep authorization downstream of every reporting path.

SECTION 06

Approach & Quality Contracts

The pipeline makes correctness observable through explicit boundaries, keys and testable invariants.

1. Registry-first routing

Client code, account code, source key and schema are resolved from PostgreSQL before a pipeline starts.

2. Idempotent Bronze

dlt merge keys prevent duplicate records; entity-level runs keep partial failures visible and resumable.

3. Declared grain

Every Silver model defines its business key and row grain before downstream metrics are allowed.

4. Reusable macros

One transformation macro handles an entity; thin account stubs supply routing rather than duplicated SQL.

5. Quality severity

Uniqueness, not-null and relationships fail builds; legitimate edge cases remain visible as warnings.

6. Privacy by default

Customer PII is excluded unless explicitly justified, and raw secrets never enter source control.

A build is evidence, not a marketing label

The project documentation separates code present, local test evidence, live database verification, and end-to-end reporting. This avoids calling a scaffold 'production-ready' and keeps unfinished Gold marts, scheduling, and live dashboards visible as remaining work.

SECTION 07

Code Spotlight

Two small patterns carry most of the platform's safety and scalability.

Fail-closed tenant routing *ingestion/pipelines/shopify/bronze.py*

```
def assert_account_isolation(account):
    expected_suffix = '_' + account.client_code
    if not account.schema_name.endswith(expected_suffix):
        raise RuntimeError('refusing cross-tenant load')
    if not account.source_key.endswith('_' + account.account_code):
        raise RuntimeError('account/source mismatch')

account = resolve_account(conn, domain)
assert_account_isolation(account)
```

Thin dbt account stub *dbt/models/staging/...*

```
{% set account_code = 'a001' %}
{% set acct = (var('shopify_accounts')
   | selectattr('account_code', 'equalto', account_code)
   | first) %}
{{ config(schema=acct['schema'],
          alias='silver_shopify_' ~ account_code ~ '_orders') }}
{{ shopify_orders_silver(acct['source'], account_code) }}
```

The first pattern blocks cross-client writes before dlt receives a dataset name. The second keeps transformation logic centralized: onboarding changes metadata and a thin stub, not the SQL contract.

SECTION 08

Verified Delivery

The current milestone is complete at Silver, with explicit evidence and explicit limits.

Source	Silver views	Data tests	Build result
Shopify	63	608	PASS - 0 errors
Meta Ads	53	412	PASS - 0 errors
Klaviyo	53	330	PASS - 0 errors
Instagram	5	40	PASS - 0 errors
Noon	4	30	PASS - 0 errors
TOTAL	**178**	**1,420**	**Current Bronze coverage**

The counts combine the documented source-specific production builds. Tests cover grain uniqueness, required fields, accepted values, parent-child relationships, currency consistency, and source-specific rules. One legitimate oversold Shopify inventory condition is preserved as a warning rather than erased.

Current boundary - deliberately not overstated

Intermediate and Gold marts are pending business definitions. The reporting UI still uses demo figures; live reporting APIs, scheduled orchestration, alerting, and complete end-to-end client reporting remain future work. Silver views currently evaluate from Bronze at query time.

SECTION 09

Outcome & Next Stage

A governed analytics foundation that makes the next reporting layer faster and safer to build.

GrowMore Reporting now has a repeatable route from heterogeneous external APIs to tenant-scoped, analytically consistent Silver data. The architecture prevents destination selection from drifting into application input, makes ingestion outcomes auditable, and keeps transformation logic reusable across accounts.

The strongest result is not a single dashboard. It is the platform contract underneath every future dashboard: stable identifiers, documented grains, preserved currencies, explicit privacy choices, and quality tests that fail before bad data becomes a KPI. This creates a reliable base for the next phase - Intermediate logic, Gold marts, scheduled publication, and authorized live reporting.

LET'S TALK

Mudasir Shafiq

Data & AI Engineer

Email chmudasir.fr@gmail.com

Stack Python/dlt | dbt | PostgreSQL | FastAPI | Redis | Next.js | Docker